AI-BASED NETWORK INTRUSION DETECTION SYSTEM

Scenario-Aware Evaluation of Machine-Learning-Based Network Intrusion Detection

CIC-IDS2017 • Machine Learning • Network Security

Research Project Report

Dataset: **CIC-IDS2017**
Model: **Random Forest (Experiment H)**

Prepared by

Hadiya Khan

GitHub: https://github.com/khhadiya-09/AI-NIDS

Live demo: https://ai-nids-cybersecurity.streamlit.app/

Draft for review; figures and results must be verified before formal submission.

ABSTRACT

Machine-learning-based network intrusion detection systems (NIDS) classify network flows as benign or malicious, but measured performance can be misleading when test data resembles training data or fails to represent unseen attack scenarios. This project develops an AI-based NIDS using CIC-IDS2017 and studies how attack representation affects generalization and minority-class detection. The workflow includes dataset auditing, duplicate-aware splitting, binary and multiclass classification, a held-out DDoS scenario, controlled DDoS exposure experiments, Bot-class analysis, and Random Forest versus XGBoost comparison. The final seven-class Random Forest achieved 98.96% accuracy and 96.01% macro F1, with 94.40% Bot recall and 72.60% Bot precision. In a separate unseen-DDoS experiment, binary recall was 63.50%; a controlled exposure experiment reported 99.79% recall with 1% representative DDoS exposure. These results are specific to the dataset and experimental protocol and are not guarantees of real-world detection performance.

CONTENTS

Abstract		**i**
1	INTRODUCTION	
2	PROBLEM STATEMENT	
3	DATASET AND PREPROCESSING	
4	METHODOLOGY	
5	EXPERIMENTAL DESIGN	
6	RESULT AND DISCUSSION	
7	DEPLOYMENT AND DASHBOARD	
8	LIMITATION AND FUTURE WORK	
9	CONCLUSION	
10	REFERENCE	

INTRODUCTION

A network intrusion detection system examines network activity and identifies patterns associated with benign use or potentially malicious behaviour. Machine learning can learn such patterns from labelled flow records, but performance depends on data quality, class representation, and the relationship between training and deployment traffic.

A random train/test split is useful but can overestimate generalization when highly similar records occur in both sets. A model may perform well on traffic resembling training examples while struggling with a new attack scenario. This project therefore combines conventional metrics with duplicate-aware and scenario-aware experiments.

1.1 Research question

How does the representation of attack traffic in training data affect the generalization and minority-class detection performance of machine-learning-based network intrusion detection systems?

1.2 Objectives

- Audit CIC-IDS2017 for class imbalance, duplicate rows, non-finite values, and constant features.
- Build binary and seven-class experiments with leakage-conscious data separation.
- Measure performance when a complete DDoS capture is withheld from training.
- Quantify the effect of limited representative DDoS exposure.
- Analyze Bot detection and its false-positive trade-off.
- Compare Random Forest and XGBoost and select a deployment model using class-specific objectives.
- Deploy a CSV-based Streamlit dashboard for inference and evaluation.

PROBLEM STATEMENT

The project investigates whether high scores under one split translate to performance on a scenario absent from training, and how minority-class representation affects recall and false positives. The implemented application is an offline, flow-level CSV demonstration. It is not a live packet-capture sensor and does not independently confirm malicious activity in a production network.

2.1 BACKGROUND

CIC-IDS2017 provides labelled benign and attack traffic for intrusion-detection research and includes multiple attack types and capture periods (Sharafaldin, Lashkari, & Ghorbani, 2018). Random Forest combines predictions from multiple decision trees and can model nonlinear patterns in tabular data (Breiman, 2001). XGBoost is a scalable gradient-boosted tree system for structured learning tasks (Chen & Guestrin, 2016).

Class imbalance is important in NIDS evaluation: high accuracy can coexist with poor recall for a minority attack. This project therefore reports precision, recall, F1, macro averages, confusion matrices, and per-class metrics alongside accuracy. This is a concise background section rather than an exhaustive systematic literature review; additional verified studies should be added before formal academic submission.

DATASET AND PREPROCESSING

The audit covered eight CIC-IDS2017 MachineLearningCSV files: 2,830,743 rows, 78 candidate numeric features, a label column, and 15 original labels. BENIGN represented approximately 80.30% of rows. Very rare original labels included Heartbleed (11 records), SQL Injection (21), and Infiltration (36).

Audit item	Observed result
Files audited	8
Rows	2,830,743
Columns	79 (78 candidate features + label)
Original labels	15
Exact duplicate row occurrences	308,381
Rows with NaN occurrences	1,358
Rows with infinity occurrences	2,867
Constant zero features removed	8

4.1 Seven-class label mapping

The primary multiclass experiment consolidated labels into BENIGN, Bot, Brute Force, DDoS, DoS, PortScan, and Web Attack. DoS subtypes were combined into DoS; brute-force labels into Brute Force; and web-attack subtypes into Web Attack. Infiltration and Heartbleed were excluded from the primary evaluation because their support was extremely small.

Class	Rows after mapping
BENIGN	2,273,097
DoS	252,661
PortScan	158,930
DDoS	128,027
Brute Force	13,835
Web Attack	2,180
Bot	1,966

Exact duplicate rows were grouped by row hash for relevant splits. The recorded multiclass index had no hash group with conflicting mapped labels. Eight constant zero features were removed. Non-finite values were imputed using medians learned from training data only; raw files were not overwritten. Unusual negative sentinel values were not indiscriminately removed. These controls reduce specific leakage risks but do not eliminate all sources of dataset dependence or bias.

METHODOLOGY

The workflow consisted of dataset auditing, label consolidation, removal of constant features, training-only median imputation, leakage-aware splitting, model training, metric-based evaluation, and packaging for inference. The Random Forest model was selected for the dashboard's seven-class inference package. XGBoost and Logistic Regression were included in comparative experiments.

The evaluation used multiple protocols rather than treating all results as directly interchangeable:

- Binary classification: benign versus attack traffic, with a random group split and a dummy majority baseline.
- Held-out DDoS scenario: a complete Friday-Afternoon-DDoS capture was withheld to probe generalization to that scenario.
- DDoS exposure ablation: selected proportions of DDoS groups were introduced into training to examine the effect of exposure.
- Seven-class classification: seven consolidated labels, group-aware separation, and a Bot decision threshold selected on validation data and locked before the untouched test evaluation.

Metrics are reported within their corresponding experiment. Scores from different splits, class mappings, and exposure protocols should not be compared as if they came from one identical test set.

.

EXPERIMENTAL DESIGN

6.1 Binary random group split

The binary experiment separated BENIGN from ATTACK and used row-hash groups. The recorded split contained 2,017,889 unique training groups and 504,473 test groups, with zero exact hash overlap. The test set was approximately 80.24% BENIGN and 19.76% ATTACK.

6.2 Unseen-DDoS scenario

The complete Friday-WorkingHours-Afternoon-DDos capture was excluded from training. The model was trained on the other seven files and evaluated on the withheld capture. This scenario-aware test asks a different question from a random split over the same collection.

6.3 Controlled DDoS exposure

Selected representative DDoS groups were introduced at 1%, 2%, and 5% exposure levels, followed by evaluation on the scenario-aware test. These outcomes depend on the specific sampling protocol and should not be interpreted as a universal minimum amount of training data.

6.4 Multiclass Bot evaluation

A leakage-controlled validation split was used to select a Bot threshold of 0.50, which was then locked for the untouched test evaluation. Random Forest and XGBoost were compared. Random Forest was retained because Bot recall was a key project objective, even though XGBoost achieved higher aggregate accuracy and macro F1.

RESULTS AND DISSCUSSION

7.1 Binary random-split results

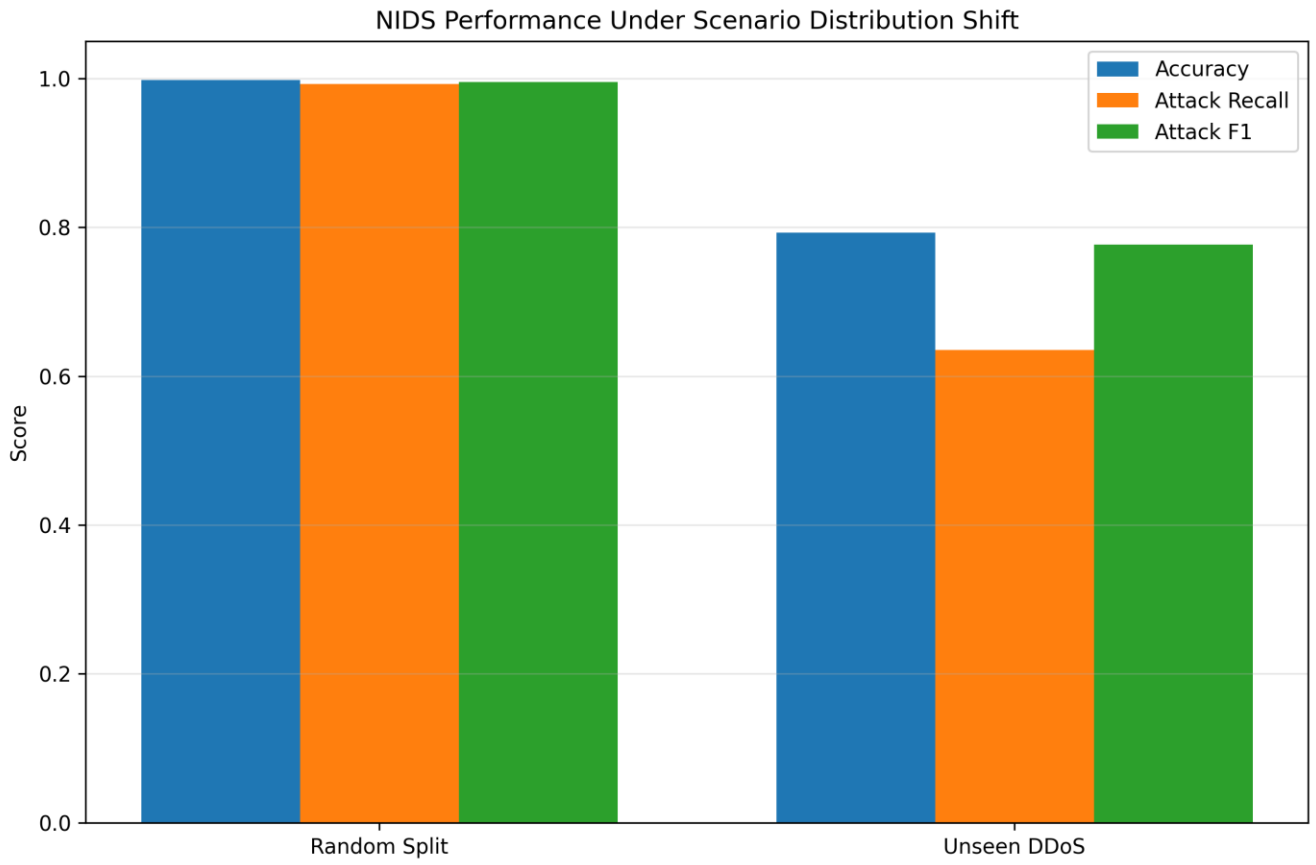
Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Dummy majority	80.24%	0.00%	0.00%	0.00%	—	—
Logistic Regression	85.88%	58.84%	94.97%	72.66%	96.74%	88.39%
Random Forest	99.80%	99.73%	99.26%	99.49%	99.97%	99.82%

Random Forest performed extremely well under the random group split. Logistic Regression had high recall but lower precision and did not converge within the recorded 100 iterations. The gap between random-split and scenario-aware performance demonstrates why one aggregate score is not sufficient.

7.2 Unseen-DDoS generalization

Metric	Random Forest
Accuracy	79.27%
Precision	99.91%
Recall	63.50%
F1	77.65%
ROC-AUC	93.79%
PR-AUC	95.01%

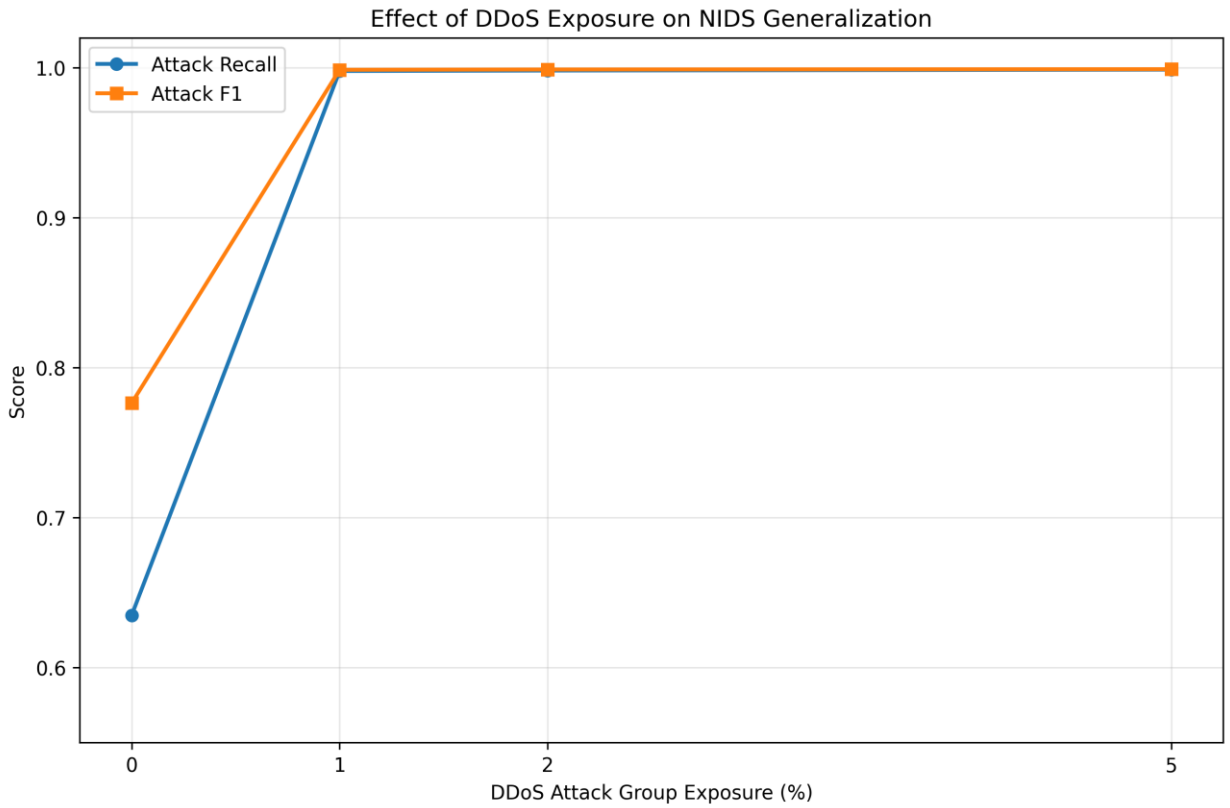
On the withheld DDoS capture, recall was 63.50%, so 36.50% of attack-positive records were missed under the experiment's binary decision rule. Precision remained high. This is evidence of a generalization gap for this particular held-out scenario.



7.3 DDoS exposure ablation

Representative DDoS exposure	Recall
0%	63.50%
1%	99.79%
2%	99.82%
5%	99.88%

Adding 1% representative DDoS exposure was associated with a 36.29 percentage-point increase in recall, from 63.50% to 99.79%. Further increases produced smaller gains in this experiment. The result supports the importance of representative attack coverage, but depends on the capture and group-selection protocol.



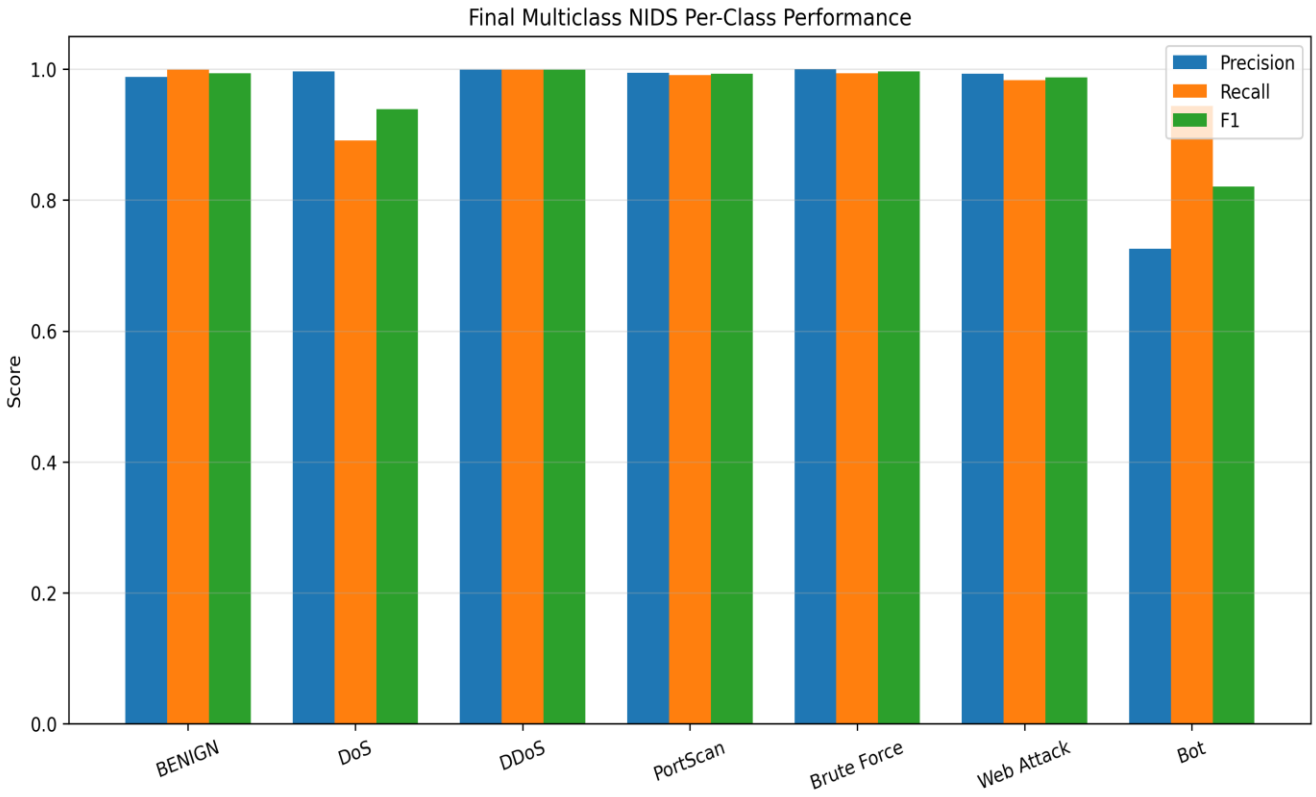
7.4 Final multiclass performance

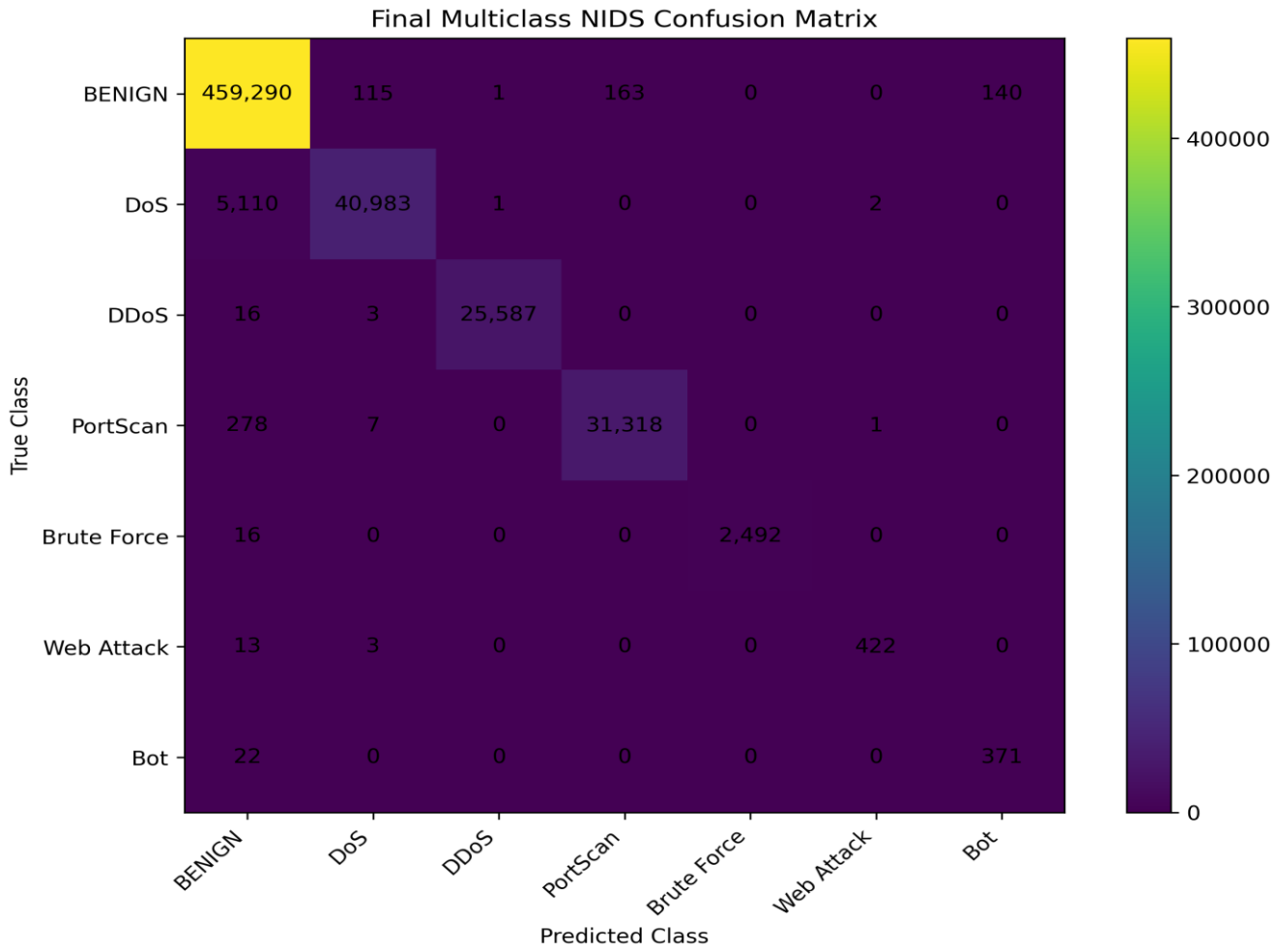
Metric	Experiment H Random Forest
Accuracy	98.96%
Macro Precision	95.72%
Macro Recall	97.05%
Macro F1	96.01%
Weighted F1	98.93%

Class	Precision	Recall	F1
BENIGN	98.83%	99.09%	99.36%
DoS	99.69%	89.11%	93.91%
DDoS	99.92%	99.93%	99.96%
PortScan	99.48%	99.10%	99.29%

Brute Force	100.00%	99.38%	99.68%
Web Attack	99.29%	98.35%	98.78%
Bot	72.60%	94.40%	82.08%

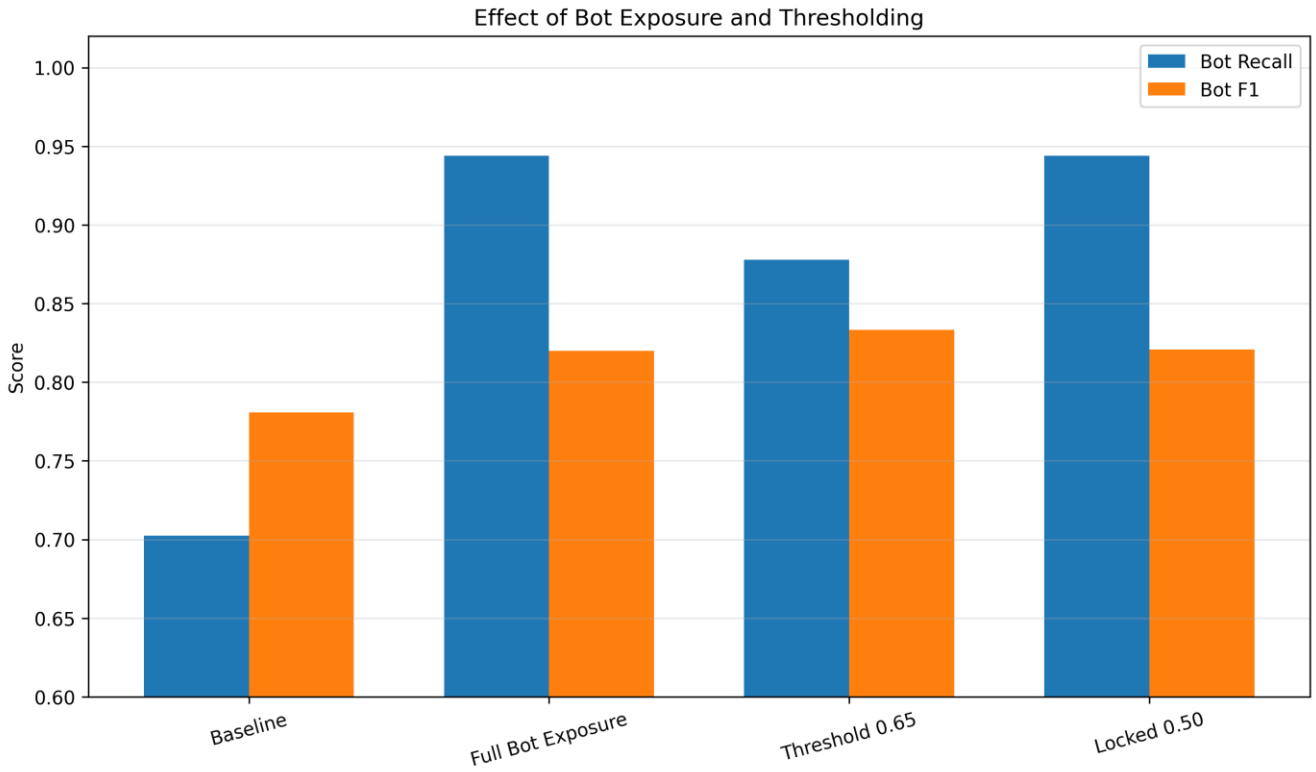
The final seven-class model achieved 98.96% accuracy and 96.01% macro F1. Performance varied by class. Bot recall was 94.40%, but precision was 72.60%, indicating false-positive costs. DoS recall was 89.11%; the recorded confusion matrix contained 5,110 DoS samples classified as BENIGN. These class-specific errors would be obscured by accuracy alone.





7.5 Bot minority-class analysis

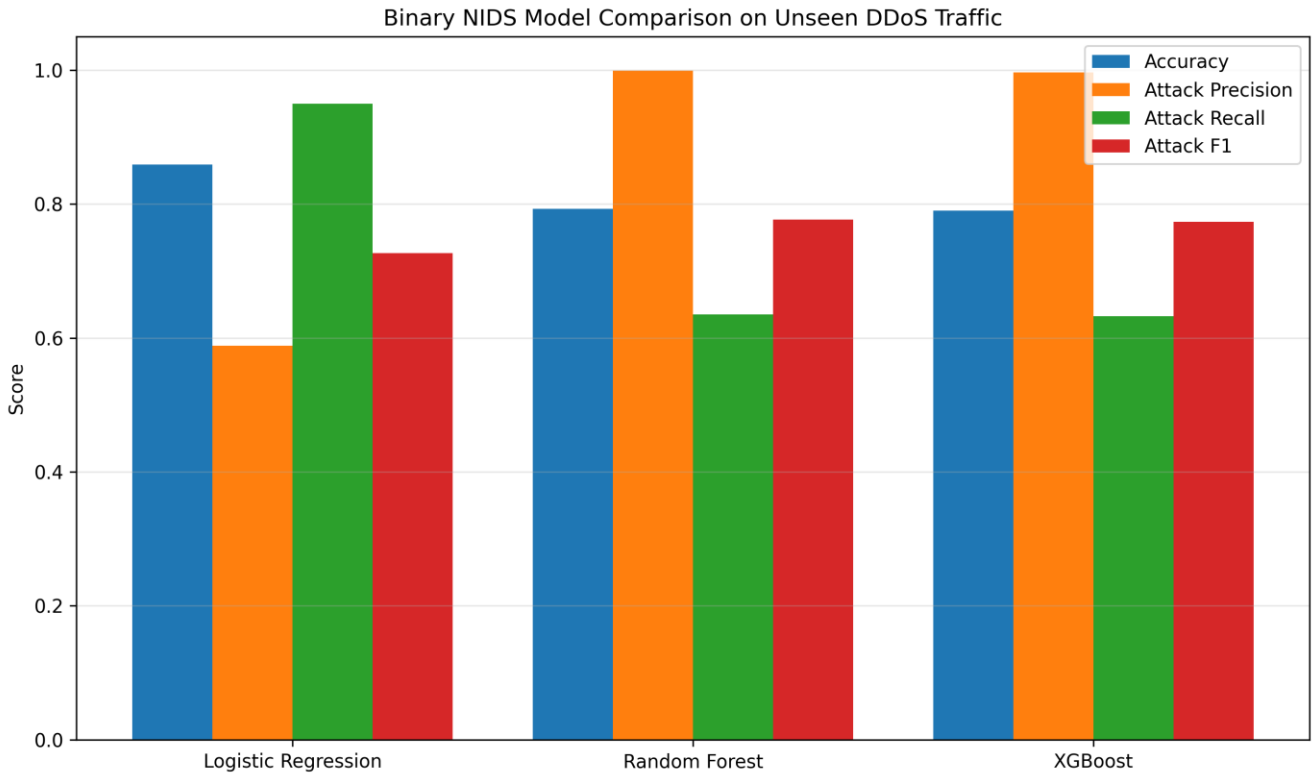
The model detected 371 of 393 Bot samples, corresponding to 94.40% recall, with 72.60% precision and 82.08% F1. Increased Bot representation during training was associated with improved recall but also benign flows classified as Bot. A threshold of 0.50 was selected on validation data and locked before test evaluation; an exploratory threshold chosen from the test set was not used as the final selection.



7.6 Random Forest versus XGBoost

Model	Accuracy	Macro F1	Bot Recall	Bot F1
Random Forest (selected)	98.96%	96.01%	94.40%	82.08%
XGBoost (standard)	99.87%	96.99%	78.63%	82.07%
XGBoost (Bot threshold 0.30)	99.87%	96.95%	82.95%	81.81%

XGBoost achieved higher aggregate accuracy and macro F1 in the recorded comparison, but lower Bot recall than the selected Random Forest. Random Forest was retained because the project prioritized Bot sensitivity while maintaining strong overall performance; this is a goal-dependent selection, not a claim that Random Forest is universally superior.


Binary NIDS Model Comparison on Unseen DDoS Traffic
Accuracy
Attack Precision
Attack Recall
Attack F1
Score
1.0
0.8
0.6
0.4
0.2
0.0
Logistic Regression
Random Forest
XGBoost

DEPLOYMENT AND DASHBOARD

The model package includes the trained Random Forest classifier, ordered feature metadata, preprocessing medians, and model metadata. A verification script checks the model type, 70 expected input features, class labels, and preprocessing configuration. The Streamlit dashboard supports CSV upload, seven-class predictions, prediction summaries, visualizations, evaluation when ground-truth labels are present, filtering, and CSV export.

GitHub repository: https://github.com/khhadiya-09/AI-NIDS

Live dashboard: https://ai-nids-cybersecurity.streamlit.app/

The deployed application is a CSV-based demonstration, not a live packet-capture system. Predictions are model outputs and must not be described as confirmed incidents without independent validation.

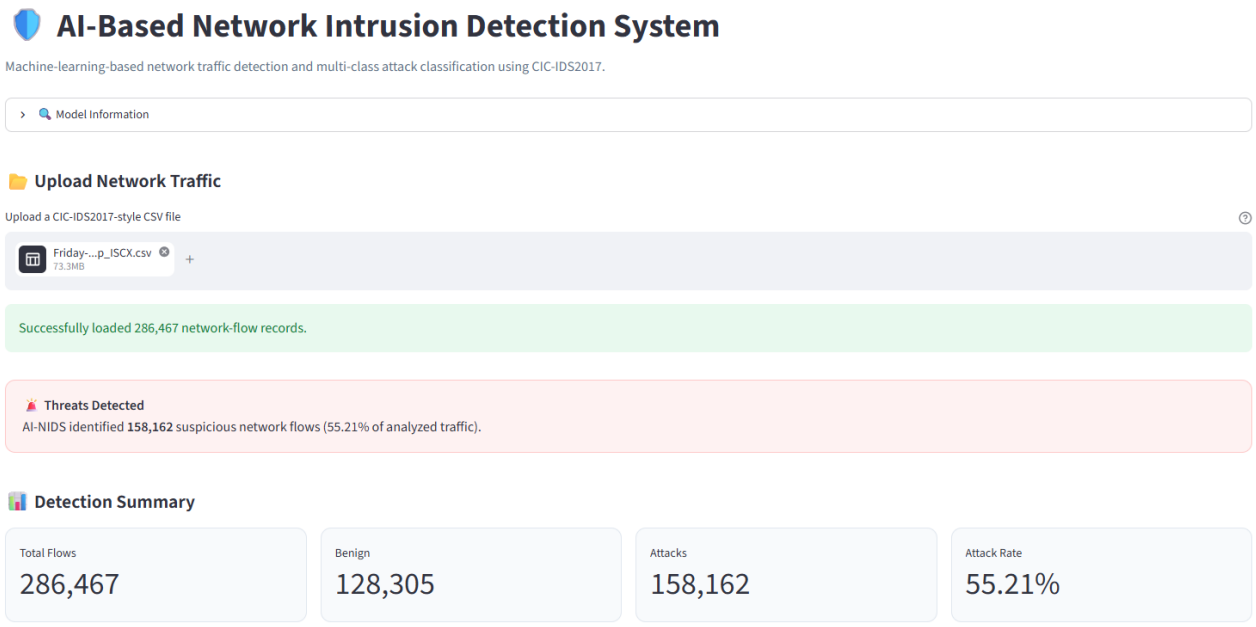
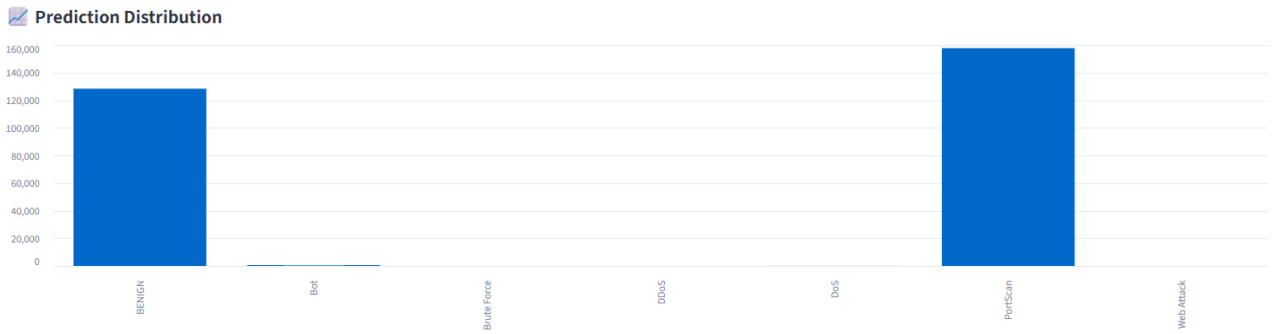


Figure 1. AI-NIDS Dashboard Overview and Threat Detection Summary



Attack Breakdown

Prediction	Detected Flows	Percentage
PortScan	157652	99.68
Bot	433	0.27
DoS	75	0.05
Web Attack	1	0
DDoS	1	0

Figure 2. Predicted Traffic Distribution and Attack Breakdown

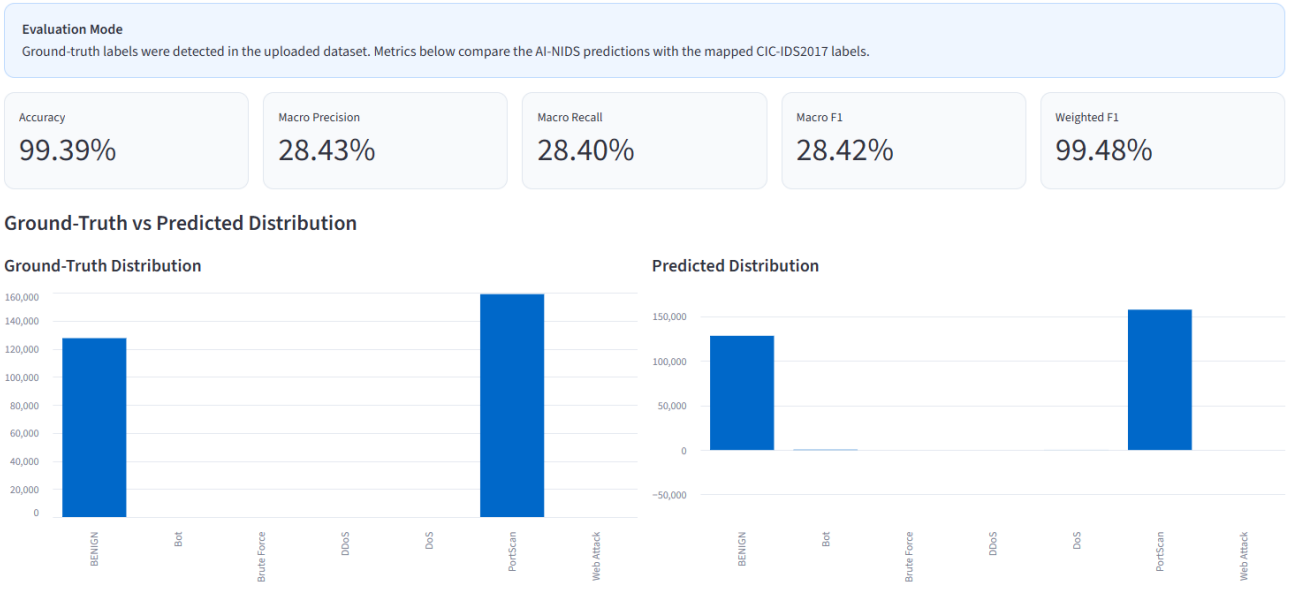


Figure 3. Evaluation Metrics and Ground-Truth versus Predicted Distribution

Per-Class Performance

Class	Precision	Recall	F1 Score	Support
BENIGN	99.04	99.64	99.34	127537
Bot	0	0	0	0
Brute Force	0	0	0	0
DDoS	0	0	0	0
DoS	0	0	0	0
PortScan	100	99.2	99.6	158930
Web Attack	0	0	0	0

Precision, Recall and F1 are shown as percentages. Support represents the number of actual samples belonging to each class.

Figure 4. Per-Class Classification Performance

Confusion Matrix

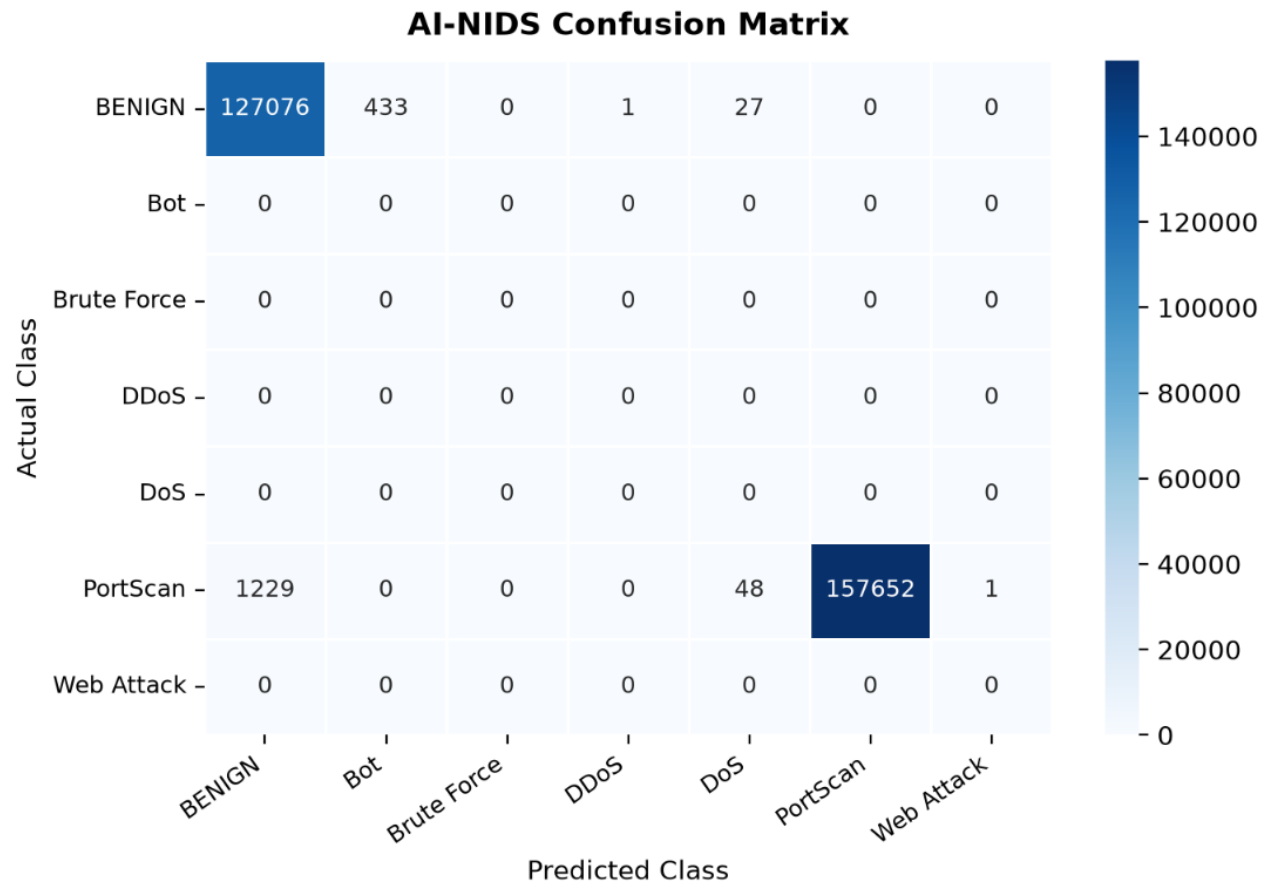
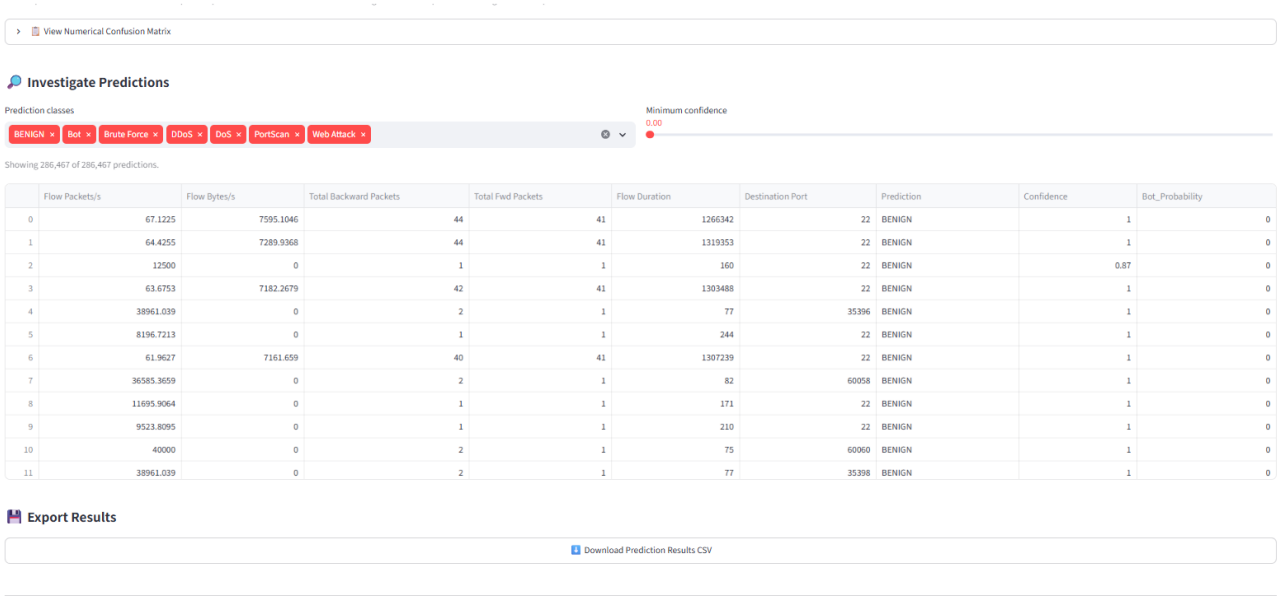


Figure 5. Dashboard Confusion Matrix Visualization



	Flow Packets/s	Flow Bytes/s	Total Backward Packets	Total Fwd Packets	Flow Duration	Destination Port	Prediction	Confidence	Bot_Probability
0	67.1225	7595.1046	44	41	1266342	22	BENIGN	1	0
1	64.4255	7289.9368	44	41	1319353	22	BENIGN	1	0
2	12500	0	1	1	160	22	BENIGN	0.87	0
3	63.6753	7182.2679	42	41	1303488	22	BENIGN	1	0
4	38961.039	0	2	1	77	35396	BENIGN	1	0
5	8196.7213	0	1	1	244	22	BENIGN	1	0
6	61.9627	7161.659	40	41	1307239	22	BENIGN	1	0
7	36585.3659	0	2	1	82	60058	BENIGN	1	0
8	11695.9064	0	1	1	171	22	BENIGN	1	0
9	9523.8095	0	1	1	210	22	BENIGN	1	0
10	40000	0	2	1	75	60060	BENIGN	1	0
11	38961.039	0	2	1	77	35398	BENIGN	1	0

Figure 6. Prediction Investigation and CSV Export

LIMITATIONS AND FUTURE WORK

- Experiments use CIC-IDS2017; performance may differ on newer, encrypted, cross-network, or independently collected traffic.
- Infiltration and Heartbleed were excluded from the primary seven-class evaluation because of very low support.
- The label mapping merges subtypes and therefore loses subtype-level detail.
- Exact-hash grouping reduces duplicate leakage but does not eliminate all temporal, host, or scenario dependence.
- The unseen-DDoS result represents one held-out capture and should not be generalized to every DDoS attack.
- The exposure ablation should be replicated across random seeds and additional held-out scenarios.
- Bot detection has a precision/recall trade-off that can create investigation overhead.
- The dashboard performs offline CSV inference rather than real-time packet capture.

Future work should include cross-dataset testing, repeated scenario-aware experiments, probability calibration, validation-driven class thresholds, model-drift monitoring, explainability, and a separately engineered live-traffic pipeline.

CONCLUSION

This project developed and evaluated a machine-learning-based NIDS using CIC-IDS2017. Strong random-split results did not guarantee comparable performance when a complete DDoS capture was unseen during training. A controlled exposure study showed that a small representative subset of DDoS traffic was associated with a substantial recall improvement under the specified protocol. Multiclass results further showed that minority-class representation and decision thresholds affect the balance between detection and false positives. Random Forest was selected for deployment because it provided higher Bot recall than the compared standard XGBoost model. The findings support leakage-conscious, scenario-aware evaluation and class-specific reporting; conclusions remain bounded by the dataset and experimental setup.

REFERENCES

[1] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 108–116. https://doi.org/10.5220/0006639801080116

[2] Breiman, L. (2001). Random Forests. Machine Learning, 45, 5–32. https://doi.org/10.1023/A:1010933404324

[3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

[4] Scikit-learn developers. (n.d.). scikit-learn documentation. https://scikit-learn.org/

[5] Streamlit, Inc. (n.d.). Streamlit documentation. https://docs.streamlit.io/